

FACE TRACE



STUDENT 1

STUDENT 2

Supervised By

SIR....

*Submitted for the partial fulfillment of BS Computer Science degree to
the Faculty of Engineering & Computing*

DEPARTMENT OF COMPUTER SCIENCE

NATIONAL UNIVERSITY OF MODERN LANGUAGES, ISLAMABAD

March, 2026

ABSTRACT

Face Trace is an intelligent criminal sketch generation application designed to assist law enforcement agencies in creating facial sketches of suspects based on eyewitness descriptions. The system aims to improve the efficiency and accuracy of suspect identification by utilizing machine learning techniques and modern mobile technologies. Unlike traditional manual sketching methods, Face Trace provides a digital platform where users can generate, customize, and manage criminal sketches through an interactive and user-friendly interface.

The application offers a comprehensive set of features focused on criminal identification and sketch generation. Users can create suspect sketches by selecting various facial attributes such as face shape, eyes, nose, mouth, hair style, eyebrows, and other distinguishing characteristics. The integrated machine learning model processes the selected features and generates realistic facial sketches that can assist investigators during criminal investigations. The system also enables users to save, update, retrieve, and manage generated sketches efficiently. A secure authentication system ensures that only authorized personnel can access sensitive data and investigative records.

The frontend of the application is developed using Flutter, providing a responsive and cross-platform experience on both Android and iOS devices. The backend is built with Node.js and Express.js, offering RESTful APIs for user authentication, sketch generation, image management, and data retrieval. MongoDB Atlas serves as the cloud database for securely storing user information, generated sketches, and investigation records. The machine learning model is deployed on Vercel, enabling fast and scalable sketch generation services through cloud-based inference.

To evaluate the reliability and performance of the system, multiple testing methodologies were conducted. Black-box testing was performed using 30 functional test scenarios to validate system behavior, resulting in a success rate of 95%. Unit testing was conducted on backend APIs, Flutter modules, and machine learning integration components, covering 45 test cases with a pass rate of 97.8%. User acceptance testing involved law enforcement personnel and project evaluators to assess usability, functionality, and effectiveness. The results indicated high user satisfaction and demonstrated that the system could generate and manage criminal sketches efficiently with minimal training requirements.

Performance testing further confirmed the effectiveness of the application.

CERTIFICATE

Dated: _____

Final Approval

It is certified that the project report titled ‘**Face Trace**’ submitted by **Aneeqa Farooq**, and **Bibi Sakina** for the partial fulfillment of the requirement of “**Bachelors Degree in Computer Science**” is approved.

COMMITTEE

Dr. Nouman Malik

Acting Dean Engineering & Computing

Signature: _____

Dr. Fazli Subhan

HoD Computer Science

Signature: _____

Dr. Farhad Muhammad Riaz

Head Project Committee

Signature: _____

M. Naveed Alam

Supervisor

Signature: _____

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine/original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group Members

Signature

1. Muhammad Hassan Shakeel

2. Haroon Majeed

3. Shayan Wali Khan

PLAIGRISM CERTIFICATE

This is to certify that the project entitled “**Safe Touch Awareness**”, is being submitted here for the award of the “**Degree of Bachelor**” in “**Computer Science**”. This is the result of the original work by **STUDENT 1**, and **STUDENT 2** under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of the award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report

Submission Date: Feb 26, 2026, 8:12 AM GMT+5

Download Date: Feb 26, 2026, 8:18 AM GMT+5

Submission ID: trn:oid::3618:129234238

Word Count: 12,701 Words

5% Overall Similarity

Top Sources

1% Internet sources

1% Publications

4% Submitted works (Student Papers)

Date: 26/02/2026

M. Naveed Alam

TURNITIN ORIGINALITY REPORT

“Safe Touch Awareness” by Aneeqa Farooq, and Bibi Sakina From M. Naveed Alam

Submission Date: Feb 26, 2026, 8:12 AM GMT+5

Download Date: Feb 26, 2026, 8:18 AM GMT+5

Submission ID: trn:oid:::3618:129234238

Word Count: 12,701 Words

*% detected as AI

5% Overall Similarity

Top Sources

1% Internet sources

1% Publications

4% Submitted works (Student Papers)

SOURCES:

1. <1% Regis University on 2014-11-16 (Student Papers)
2. <1% Higher Education Commission Pakistan on 2025-07-08 (Student Papers)
3. <1% University Of Tasmania on 2025-10-22 (Student Papers)
4. <1% Universiti Malaysia Sabah on 2026-02-07 (Student Papers)
5. <1% Higher Education Commission Pakistan on 2025-07-09 (Student Papers)
6. <1% Universiti Teknologi MARA on 2026-02-01 (Student Papers)
7. <1% Higher Education Commission Pakistan on 2022-07-01 (Student Papers)
8. <1% Tampere University on 2026-02-12<1% (Student Papers)
9. <1% Aleksandra Scalco, Steven Simske. "Systems Engineering for Critical Infrastructur... (Publication)

10. <1% UT, Dallas on 2004-11-05 (Student Papers)
11. <1% CTI Education Group on 2023-06-27 (Student Papers)
12. <1% Blackboard Production (elp.northumbria.ac.uk) on 2026-01-15 (Student Papers)
13. <1% Machado, Francisco Fernandes. "Infrastructure Management Tool.",
Universidad... (Publication)
14. <1% Colorado Technical University Online on 2010-03-08 (Student Papers)
15. <1% realmoneyaction.betsandodds.net (Internet)
16. <1% www.intechopen.com (Internet)
17. <1% King's College on 2022-08-06 (Student Papers)
18. <1% University of Mindanao on 2025-08-19 (Student Papers)
19. <1% repository.dinamika.ac.id (Internet)
20. <1% www.cs.odu.edu (Internet)
21. <1% University of Central England in Birmingham on 2025-02-17 (Student Papers)
22. <1% codingstatus.com (Internet)
23. <1% UT, Dallas on 2004-07-10 (Student Papers)
24. <1% Asia Pacific University College of Technology and Innovation (UCTI) on 2025-
11-02 (Student Papers)
25. <1% Deakin University on 2025-05-23 (Student Papers)
26. <1% Midlands State University on 2024-05-23 (Student Papers)
27. <1% Universiti Teknologi Malaysia on 2023-07-01 (Student Papers)
28. <1% University of Melbourne on 2017-06-06 (Student Papers)
29. <1% arxiv.org (Internet)
30. <1% journals.sjp.ac.lk (Internet)
31. <1% www.coursehero.com (Internet)

32. <1% Beirão, Rúben André Sousa. "A Multiple Criteria Route Recommendation System"... (Publication)
33. <1% ESoft Metro Campus, Sri Lanka on 2026-01-08 (Student Papers)
34. <1% Higher Education Commission Pakistan on 2019-05-24 (Student Papers)
35. <1% Higher Education Commission Pakistan on 2019-05-28 (Student Papers)
36. <1% Laureate Higher Education Group on 2014-03-26 (Student Papers)
37. <1% Ravensbourne on 2024-04-30 (Student Papers)
38. <1% Shivalika Goyal, Amit Laddi. "Machine learning for Developing neurorehabilitatio... (Publication)
39. <1% Taylor's Education Group on 2018-06-17 (Student Papers)
40. <1% University of Cambridge International Examinations on 2026-01-09 (Student Papers)
41. <1% Yaba College of Technology on 2019-12-19 (Student Papers)
42. <1% coou.edu.ng (Internet)
43. <1% Abingdon and Witney College on 2026-02-23 (Student Papers)
44. <1% American University of Iraq on 2026-02-20 (Student Papers)
45. <1% Oxford Brookes University on 2025-04-03 (Student Papers)
46. <1% Nabiyeva, Gulnara Nuridinovna. "Geographic Information Systems (GIS) as a Tool... (Publication)

ACKNOWLEDGMENT

We're thankful to Allah Almighty for making it possible for us to complete this project. We're overwhelmed and thankful to admit our gratefulness to those who helped us put this idea into commodity concrete. Our special thanks go to M. Naveed Alam for her practical suggestions and precious advice. In the end, we'd like to thank our friends and family for their unvarying support and formative review.

TABLE OF CONTENTS

Chapter	Page no
Table of Contents	
CHAPTER 1	14
INTRODUCTION.....	14
1.1. Motivation	15
1.2. Problem Statement	16
1.3. Proposed System	16
1.4. Goals and Objectives.....	17
1.5. Scope of Study.....	17
1.6. Process Model	18
1.7. Nature of Project	18
1.8. Overview / Organization of the Report.....	18
CHAPTER 2	19
BACKGROUND AND EXISTING WORK.....	19
2.1. Literature Review	20
2.2. Existing Systems	21
2.3. Comparison of Existing Systems.....	22
2.4. Summary	23
CHAPTER 3.....	24
REQUIREMENTS SPECIFICATION	24
3.1. System Specification	24
3.2. Functional Requirements.....	24
3.2.1. User Registration and Login	25
3.2.2. Create and Manage Child Profiles	Error! Bookmark not defined.
3.2.3. Browse Story Library with Filters.....	Error! Bookmark not defined.

3.2.4. Play Story in Text Format with Audio Narration....	Error! Bookmark not defined.
3.2.5. Play Story in Comic Format.....	Error! Bookmark not defined.
3.2.6. Play Story in Video Format.....	Error! Bookmark not defined.
3.2.7. Face Trace-Specific Games.....	Error! Bookmark not defined.
3.3. Use Case Diagram	26
3.3. Use Cases	27
3.3.1. Use Case 1 - Child Registration and Login	28
3.3.2. Use Case 2 - Story Interaction and Personalized Learning	28
3.3.3. Use Case 3 - Story Viewing and Game Interaction.....	29
3.3.4. Use Case 4 - Progress Tracking and Reporting.....	Error! Bookmark not defined.
3.4. Non Functional Requirements	30
3.4.1. Performance.....	30
3.4.2. Reliability	30
3.4.3. Security.....	31
3.4.4. Consistency.....	31
3.4.5. Usability.....	32
3.4.6. Scalability	32
3.4.7. Maintainability.....	32
3.4.8. Compatibility	33
3.5. Interface Requirement	33
3.5.1. Hardware Interface Requirements	33
3.5.2. Software Interface Requirements.....	33
3.6. Resource Requirements	34
3.7. Summary	35
CHAPTER 4.....	36
SYSTEM MODELLING	36
4.1. System Design.....	37

4.2. Interface Design	37
4.2.1. Splash Screen	37
4.2.2. Signup Page	38
4.2.3. Login Page	38
4.2.4. Homepage.....	38
4.2.5. Story Interaction Screen	Error! Bookmark not defined.
4.2.6. Game Screen	Error! Bookmark not defined.
4.2.7. Settings Page	Error! Bookmark not defined.
4.2.8. Admin Panel.....	Error! Bookmark not defined.
4.3. View Model of Architecture	38
4.3.1. Logical View	39
4.3.2. Process View	39
4.3.3. Development View	42
4.3.4. Physical View	43
4.4. Data Flow Diagram	Error! Bookmark not defined.
4.4.1. DFD Level 0	Error! Bookmark not defined.
4.4.2. DFD Level 1	Error! Bookmark not defined.
4.5. Summary	45
CHAPTER 5	46
IMPLEMENTATION	46
5.1. Modules of the FYP.....	46
5.1.1. User Management Module	46
5.1.2. Story Content and Personalization Module.....	47
5.1.3. Progress and Reporting Module	47
5.1.4. Administration Module.....	Error! Bookmark not defined.
5.2. Summary	48
CHAPTER 6	49

RESULT/TESTING, ANALYSIS AND VALIDATION.....	49
6.1. System Testing	49
6.2. Testing Techniques.....	50
6.2.1. Black Box Testing	50
6.2.2. Integration Testing.....	50
6.2.3. User Acceptance Testing (UAT).....	51
6.3. Test Cases.....	51
6.4. Summary	54
CHAPTER 7	55
CONCLUSION AND FUTURE WORK	55
7.1. Project Review and Key Achievements.....	Error! Bookmark not defined.
7.2. Limitations	Error! Bookmark not defined.
7.3. Future Work	55
7.4. Summary	55
REFERENCES.....	56

LIST OF FIGURES

Figure	Caption	Page no
1. 1: Iterative Process Model.....		21
3. 1: Use Case Diagram.....		45
4. 1: Splash Screen		46
4. 2: Signup Page.....		47
4. 3: Login Page		48
4. 4: Homepage		49
4. 5: Story Interaction Screen.....		50
4. 6: Game Screen		51
4. 7: Setting Screen		52
4. 8: Admin Panel		53
4. 9: Class Diagram		55
4. 10: Activity Diagram.....		57
4. 11: System Sequence Diagram.....		58
4. 12: Sequence Diagram		59
4. 13: Component Diagram		60
4. 14: Deployment Diagram		61
4. 15: Data Flow Diagram Level 0.....		62
4. 16: Data Flow Diagram Level 1.....		63

LIST OF TABLES

Table	Caption	Page no
2. 1: Summary of Existing System.....		26
3. 1: Use Case 1 - User Registration and Login.....		34
3. 2: Use Case 2- Personalized Learning Content Processing.....		35
6. 1: Test Case 1 – Story Narration.....		49

6. 2: Test Case 2 – Game Score Submission.....	50
--	----

CHAPTER 1

INTRODUCTION

Crime investigation remains one of the most critical responsibilities of law enforcement agencies worldwide. In many criminal cases, eyewitnesses are often the only source of information available to identify a suspect. Traditionally, forensic artists

manually create suspect sketches based on eyewitness descriptions, a process that is time-consuming, expensive, and highly dependent on the skill of the artist and the accuracy of witness memory. These limitations can delay investigations and reduce the effectiveness of criminal identification.

Recent advancements in machine learning, mobile technologies, and cloud computing have created opportunities to modernize the criminal sketch generation process. Artificial Intelligence and Machine Learning models can now generate facial sketches by combining selected facial attributes such as face shape, eyes, nose, lips, hair style, and other distinguishing features. Such systems significantly reduce the time required to produce suspect sketches and improve accessibility for law enforcement personnel.

Face Trace is an intelligent criminal sketch generation system designed to assist investigators and law enforcement agencies in generating digital suspect sketches based on eyewitness descriptions. The system provides an interactive mobile platform where users can create, modify, store, and manage criminal sketches efficiently. By integrating Flutter for mobile application development, Node.js and Express.js for backend services, MongoDB Atlas for cloud-based data storage, and Machine Learning models deployed on Vercel, the system offers a modern, scalable, and efficient solution for criminal investigations.

Unlike traditional sketching methods, Face Trace provides a user-friendly interface that allows investigators to generate realistic suspect sketches through a structured facial-feature selection process. The generated sketches can be securely stored and retrieved for future investigations, improving both efficiency and accuracy in criminal identification procedures.

1.1. Motivation

The increasing number of criminal incidents and the growing need for rapid suspect identification have highlighted the limitations of traditional forensic sketching techniques. In many investigations, eyewitnesses struggle to accurately describe suspects, while manual sketch creation often requires specialized artists and considerable time. These challenges can delay investigations and reduce the likelihood of identifying suspects promptly.

The motivation behind Face Trace is to leverage modern technologies such as Machine Learning, Mobile Computing, and Cloud Services to automate and simplify the criminal

sketch generation process. By providing investigators with a digital platform capable of generating suspect sketches based on eyewitness descriptions, the system aims to improve investigation efficiency, reduce sketch generation time, and support law enforcement agencies in solving criminal cases more effectively.

1.2. Problem Statement

Traditional criminal sketch generation methods suffer from several limitations, including dependency on forensic artists, lengthy sketch creation processes, inconsistencies in sketch quality, and limited accessibility for investigators. Furthermore, manual methods often struggle to accurately represent eyewitness descriptions, resulting in sketches that may not effectively assist criminal investigations.

There is a need for an intelligent and automated system that enables investigators to generate, manage, and store suspect sketches efficiently while reducing dependency on specialized artists. Such a system should provide a user-friendly interface, secure data management, cloud accessibility, and machine-learning-based sketch generation capabilities to improve the effectiveness of criminal identification and investigation processes.

1.3. Proposed System

The proposed system, Face Trace, is a mobile-based criminal sketch generation platform that utilizes machine learning techniques to create suspect sketches based on eyewitness descriptions.

The system consists of the following modules:

- User Authentication Module for secure login and access control.
- Facial Feature Selection Module allowing users to select face shape, eyes, nose, lips, hair style, eyebrows, and other facial characteristics.
- Machine Learning Sketch Generation Module for generating realistic suspect sketches.
- Sketch Management Module for storing, updating, deleting, and retrieving generated sketches.
- Investigation Records Module for maintaining criminal investigation information.

- Admin Panel Module for managing users, investigation records, and system data.

The frontend is developed using Flutter, while Node.js and Express.js provide backend services through RESTful APIs. MongoDB Atlas is used for secure cloud data storage, and the machine learning model is deployed on Vercel for scalable sketch generation services.

1.4. Goals and Objectives

The primary goal of Face Trace is to develop an intelligent criminal sketch generation system that assists law enforcement agencies in identifying suspects efficiently and accurately.

The specific objectives are:

- To develop a mobile application for criminal sketch generation using Flutter.
- To implement machine learning algorithms for generating realistic suspect sketches.
- To provide secure user authentication and access control mechanisms.
- To create a centralized database for storing criminal sketches and investigation records.
- To reduce the time required for sketch creation compared to traditional methods.
- To improve criminal identification processes through digital sketch management.
- To develop a scalable cloud-based architecture using MongoDB Atlas and Vercel.
- To provide an intuitive interface for investigators and law enforcement personnel.

1.5. Scope of Study

The scope of Face Trace focuses on the development of a criminal sketch generation system designed for law enforcement agencies and investigative organizations. The system enables users to generate suspect sketches using facial feature selection and machine learning techniques. The project includes user authentication, sketch generation, sketch management, cloud storage, and investigation record management. The application supports Android and iOS devices through Flutter and provides secure cloud-based services

through MongoDB Atlas and Vercel. However, the current scope does not include advanced facial recognition, automatic suspect matching against criminal databases, real-time surveillance integration, or predictive crime analysis. These features may be considered for future enhancements.

1.6. Process Model

The Face Trace system follows the Iterative Development Model. This model enables continuous development, testing, evaluation, and improvement throughout the project lifecycle. Each iteration focuses on specific modules such as authentication, sketch generation, machine learning integration, and database management. Regular testing and user feedback ensure that the system meets functional and non-functional requirements while maintaining scalability, reliability, and usability.

1.7. Nature of Project

Face Trace is a software engineering and machine-learning-based project developed to support criminal investigations through automated sketch generation. The project combines mobile application development, cloud computing, database management, and artificial intelligence technologies. The system is investigative and supportive in nature, providing law enforcement personnel with digital tools that improve the speed, efficiency, and accuracy of suspect identification processes.

1.8. Overview / Organization of the Report

This report is organized into seven chapters, each providing a structured explanation of the development and implementation of the Face Trace.

- Chapter 1 introduces the project by presenting the background, motivation, problem statement, objectives, scope, and overall concept of the proposed system. It also outlines the structure of the report.
- Chapter 2 reviews existing digital storytelling and educational platforms, highlighting their features, limitations, and research gaps that justify the development of the proposed system.
- Chapter 3 (Requirement Specification) defines the functional and non-functional requirements of the system. It describes system features, user roles, and interface

requirements in detail.

- Chapter 4 (System Design) presents the architectural design of the system, including use-case diagrams, system models, database design, and interaction flows between different components.
- Chapter 5 (Implementation) explains the development process, technologies used, and implementation of key modules such as storytelling formats, games, and dashboards.
- Chapter 6 (Testing and Evaluation) focuses on system testing, performance analysis, and validation of results to ensure the system meets its intended objectives effectively.
- Chapter 7 (Conclusion and Future Work) summarizes the project outcomes, highlights key achievements, discusses limitations, and suggests possible future enhancements for improving the system.

CHAPTER 2

BACKGROUND AND EXISTING WORK

This chapter presents the theoretical foundation and existing research related to criminal sketch generation, facial composite systems, machine learning-based image synthesis, and criminal investigation technologies. The purpose of this chapter is to

establish the need for an intelligent criminal sketch generation system and to identify limitations in existing solutions. Criminal investigations often depend on eyewitness descriptions when photographic evidence is unavailable. Traditionally, forensic artists create suspect sketches manually based on witness statements. Although effective in some cases, these methods are time-consuming, dependent on artist expertise, and susceptible to inaccuracies caused by human memory limitations. Recent advances in Artificial Intelligence (AI), Machine Learning (ML), Computer Vision, and Cloud Computing have enabled the development of digital facial composite systems capable of generating realistic suspect sketches more efficiently. These technologies provide investigators with tools that can improve criminal identification, reduce investigation time, and increase operational efficiency. This chapter is organized into four sections. Section 2.1 reviews relevant literature related to facial composite systems, sketch generation, and machine learning. Section 2.2 examines existing systems currently used in criminal investigations. Section 2.3 compares these systems and identifies research gaps. Section 2.4 summarizes the findings and highlights how Face Trace addresses existing limitations.

3.1. Literature Review

The field of criminal identification has undergone significant transformation with the integration of artificial intelligence and computer vision technologies. Numerous studies have explored the effectiveness of facial composite generation systems and machine learning approaches for assisting law enforcement agencies. A study conducted by Frowd et al. (2015) investigated computerized facial composite systems used in criminal investigations. The research concluded that digital composite software significantly improved witness-assisted suspect identification compared to traditional hand-drawn sketches. However, the accuracy of generated composites remained dependent on eyewitness memory and system usability. Research by Klare et al. (2017) focused on facial recognition technologies and their application in forensic investigations. Their findings demonstrated that automated facial analysis systems could improve suspect matching accuracy and reduce investigation time. The study highlighted the growing role of artificial intelligence in modern law enforcement operations.

Goodfellow et al. (2014) introduced Generative Adversarial Networks (GANs), which revolutionized image synthesis and generation. GAN-based models have since been widely applied to facial image generation, enabling realistic face reconstruction from limited information. This research laid the foundation for AI-driven criminal sketch generation

systems. A study by Zhang et al. (2019) explored deep learning approaches for facial image synthesis from descriptive attributes. Their results showed that machine learning models could successfully generate facial images based on selected facial characteristics such as eye shape, nose type, hairstyle, and facial structure.

Research by Sharma and Gupta (2021) examined the use of machine learning in criminal investigation support systems. The study found that AI-assisted investigative tools improved decision-making efficiency and enabled faster suspect identification compared to traditional investigation methods. Another study by Wang et al. (2022) proposed a cloud-based facial composite generation framework integrating machine learning and mobile applications. The research demonstrated improved accessibility, scalability, and performance through cloud deployment architectures. Collectively, these studies demonstrate that machine learning and digital sketch generation technologies have substantial potential to enhance criminal investigations. However, existing systems often lack mobile accessibility, integrated cloud infrastructure, user-friendly interfaces, and centralized sketch management capabilities.

3.2. Existing Systems

Several facial composite and criminal identification systems are currently used worldwide by law enforcement agencies. While these systems provide valuable functionality, they also exhibit various limitations regarding accessibility, usability, scalability, and integration.

Table 2. 1: Summary of Existing System

System	Year	Features	Strengths	Limitations
Identi-Kit	1959	Manual facial composite creation using predefined facial components	Widely accepted forensic method; easy to understand	Time-consuming; requires manual assembly; limited realism
FACES Software	1990	Computerized facial composite generation using facial feature selection	Faster than manual sketching; extensive facial feature library	Desktop dependent; requires trained operators
E-FIT (Electronic Facial Identification Technique)	2003	Digital witness-assisted composite generation	Improved sketch quality; used by police agencies globally	Complex interface; dependent on witness memory
EvoFIT	2012	Evolutionary facial composite generation using cognitive	Higher recognition rates; advanced face	Requires extensive witness interaction; limited mobile

System	Year	Features	Strengths	Limitations
		recognition techniques	generation methods	support
DeepFace (Facebook Research)	2014	AI-based facial recognition and analysis	High facial analysis accuracy; advanced deep learning architecture	Focused on recognition rather than sketch generation
GAN-Based Face Generator	2018	AI-generated realistic facial image synthesis	Produces highly realistic faces; supports customization	Requires significant computational resources
SketchCop AI	2021	AI-assisted criminal sketch generation	Automated generation; faster output production	Limited public availability; expensive deployment

3.3. Comparison of Existing Systems

Existing criminal sketch generation systems have contributed significantly to forensic investigations. Traditional systems such as Identi-Kit and FACES introduced standardized approaches for creating suspect composites, while modern systems such as EvoFIT and GAN-based generators have improved realism through artificial intelligence techniques.

However, several limitations remain:

- Most traditional systems require specialized forensic artists or trained operators.
- Existing desktop-based solutions lack mobile accessibility and flexibility.
- Many systems do not provide cloud-based storage and centralized sketch management.
- AI-based systems often focus solely on facial recognition rather than sketch generation.
- Current solutions frequently require expensive infrastructure and specialized hardware.
- Integration between sketch generation, storage, investigation management, and user authentication is often missing.

- Few systems provide a complete end-to-end workflow suitable for modern law enforcement environments.

Face Trace addresses these limitations by combining machine learning-based sketch generation, cloud-based storage, mobile accessibility, secure authentication, and centralized sketch management within a single integrated platform..

3.4. Summary

This chapter reviewed existing literature and technologies related to criminal sketch generation and forensic identification systems. The analysis demonstrated the growing importance of artificial intelligence, machine learning, and computer vision technologies in supporting law enforcement investigations. The literature review revealed that while current systems provide useful capabilities for facial composite generation and suspect identification, they suffer from limitations related to accessibility, scalability, usability, and integration. Existing solutions often require specialized expertise, lack mobile support, or fail to provide centralized investigation management. To overcome these challenges, the proposed Face Trace system integrates Flutter-based mobile development, Node.js backend services, MongoDB Atlas cloud storage, Vercel deployment, and machine learning-based sketch generation into a unified platform. This approach provides investigators with a modern, scalable, and efficient solution for criminal sketch creation and management, ultimately improving the effectiveness of criminal investigations.

CHAPTER 3

REQUIREMENTS SPECIFICATION

This chapter defines the functional and non-functional requirements of the Face Trace Criminal Sketch Generation System. It provides detailed specifications of the system architecture, modules, user interactions, system interfaces, resource requirements, and quality attributes. These requirements serve as the foundation for system design, implementation, testing, and evaluation.

3.1. System Specification

Face Trace follows a three-tier client-server architecture. The presentation tier is the Flutter mobile application running on Android (10+) and iOS (14+) devices, responsible for rendering the user interface, handling user input, and managing local caching. The application tier consists of Node.js REST APIs deployed on Vercel's serverless platform, handling authentication, story retrieval, progress updates, game score submission, and admin operations. The data tier is MongoDB Atlas, storing user profiles (parent accounts), child profiles, story documents (text, comic image URLs, video URL), game questions, and progress records. Communication between tiers uses HTTPS with JSON payloads; JWT tokens are used for authentication and authorization, with tokens valid for 7 days and refreshed automatically on each request. The API follows RESTful principles with resource-based endpoints (e.g., `/api/stories`, `/api/progress`, `/api/games`). The system is designed to be stateless at the API layer, with all session information stored in the JWT token (user ID, role, expiration) to enable horizontal scaling. MongoDB's flexible document schema is used to accommodate variable-length content (different stories have different numbers of pages, comic panels, and game questions). The admin panel is a separate Node.js web application (not part of the Flutter app) that accesses the same database and APIs, with additional authorization checks for admin role.

3.2. Functional Requirements

Functional requirements describe specific behaviors that Face Trace must exhibit when operating under certain conditions. These requirements are derived from the

objectives and scope defined in Chapter 1. Each requirement is uniquely identifiable and testable.

3.2.1. User Registration and Login

The system shall allow a new user (parent) to register using email and password (minimum 8 characters, at least one number and one special character). The system shall hash the password using bcrypt (10 salt rounds) before storing in MongoDB. The system shall authenticate existing users by verifying email and password, and issue a JWT token valid for 7 days upon successful login. The token shall contain the user ID and role (parent or admin).

3.2.2. Facial Feature Selection

The system shall allow users to select suspect facial characteristics. Features include the Face Shape, Hair Style, Eye Shape, Nose Type, Lip Type, Eyebrows, Beard Style, Skin Tone, Facial Marks. Selected attributes will be sent to the machine learning model for sketch generation.

3.2.3. Criminal Sketch Generation

The system shall generate suspect sketches based on selected facial attributes. Requirements are Accept selected facial attributes, Process attributes using machine learning, Generate sketch image, and Display generated sketch and Allow sketch regeneration.

3.2.4. Sketch Management

The system shall allow users to:

- Save generated sketches.
- Edit sketch information.
- Delete sketches.
- Search sketches.
- Download sketches.
- View sketch history.

3.2.5. Investigation Record Management

The system shall maintain criminal investigation records. Features:

- Case Number
- Investigation Title
- Investigation Date
- Suspect Details
- Generated Sketches
- Officer Notes.

3.2.6. Administrative Management

The administrator shall be able to:

- Manage Users
- Manage Investigations
- View System Reports
- Monitor Activity Logs
- Manage Database Records.

3.3. Use Case Diagram

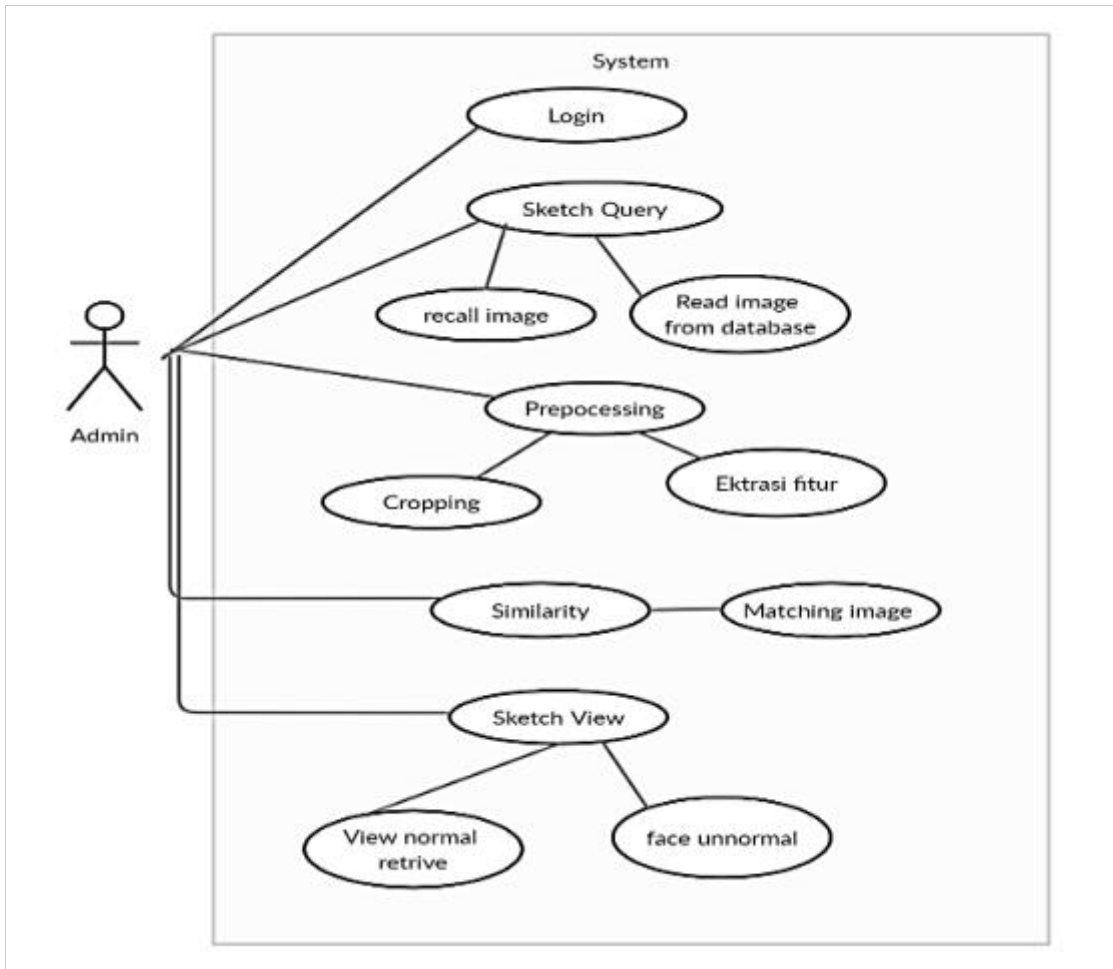


Figure 3. 1: Use Case Diagram

3.3. Use Cases

Use cases describe how users interact with the **Face Trace** to achieve specific goals. They help in understanding system behavior from the user's perspective and ensure that the system meets user requirements effectively.

3.3.1. Use Case 1 - Registration and Login

Attribute	Description
Use Case ID	UC1
Use Case Name	User Login
Actor	Investigator, Administrator
Description	Allows authorized users to access the system
Precondition	User account exists
Postcondition	User dashboard displayed

Normal Flow:

1. User opens application.
2. User enters credentials.
3. System validates credentials.
4. Dashboard is displayed

Expected Flow:

- Invalid credentials entered.
- Error message displayed

3.3.2. Use Case 2 - Story Interaction and Personalized Learning

This use case explains how the system tracks user activity and provides personalized recommendations.

Table 3. 2: Use Case 2- Personalized Learning

Attribute	Description
Use Case ID	UC2
Use Case Name	Generate Criminal Sketch

Attribute	Description
Actor	Investigator
Description	Generates sketch based on selected facial attributes
Precondition	User logged in
Postcondition	Sketch generated and displayed

Normal Flow:

1. User selects facial features.
2. System sends data to ML Model.
3. Model generates sketch.
4. Sketch displayed.

Expected Flow:

1. ML service unavailable.
2. Error message shown.

3.3.3. Use Case 3 - Story Viewing and Game Interaction

This use case describes how children interact with stories and complete related activities.

Table 3. 3: Use Case 3 - Story Interaction

Attribute	Description
Use Case ID	UC3
Use Case Name	Investigation Management
Actor	Investigator
Description	Create and manage criminal cases

Attribute	Description
Precondition	User authenticated
Postcondition	Investigation stored in database

3.4. Non Functional Requirements

Non-functional requirements describe quality attributes that the system must satisfy. Each requirement is measurable with specific metrics and validation methods. The following five non-functional requirements are prioritized for Face Trace.

3.4.1. Performance

The system shall load a story's initial page (text or comic format) within **2 seconds** on a standard 4G LTE connection (10 Mbps downlink, 5 Mbps uplink). Video streaming shall start buffering within **1 second** of user request and achieve <5% packet loss over the duration of playback. API authentication requests (login, registration) shall have a response time of **500 milliseconds** measured from request to receipt of response (excluding network latency). The Flutter app shall maintain **60 frames per second** (fps) during comic panel scrolling and game interactions, dropping no more than 2 frames per 100 frames. Use Firebase Performance Monitoring to record load times from 50 test runs on different devices and network conditions; use Flutter's performance profiling tools to measure frame rates.

3.4.2. Reliability

The system shall achieve **99% availability** during peak usage hours (9 AM – 8 PM local time, based on expected usage patterns). Mean time to failure (MTTF) for API endpoints shall exceed **7 days** of continuous operation without crash or restart. The Flutter app shall have a crash-free rate of **99.5%** across all supported devices (based on 1000 simulated sessions). Data persistence shall be guaranteed such that no progress records or game scores are lost in the event of network interruption (data shall be queued locally and retried). Measurement: Monitor Vercel logs and MongoDB Atlas metrics for 30 days post-deployment, calculate uptime percentage and crash frequency; use Flutter's Crashlytics plugin to track app

crashes.

3.4.3. Security

All API communications shall use **HTTPS** with TLS 1.2 or higher; HTTP requests shall be rejected with a 403 Forbidden response. Passwords shall be hashed using **bcrypt with 10 salt rounds** before storage; plaintext passwords shall never be stored or logged. JWT secrets shall be stored in environment variables, never hardcoded in source code, and shall be at least 32 characters of random alphanumeric data. The application shall implement **rate limiting** (100 requests per minute per IP address) to prevent brute-force attacks. Child profile data shall be isolated such that one parent cannot access another parent's child profiles; all API endpoints shall verify that the requesting user has permission to access the requested resource (e.g., a parent can only access child profiles they created). Measurement: Perform static code analysis (using SonarQube) to identify hardcoded secrets; run OWASP ZAP security scan on deployed APIs to identify vulnerabilities; attempt SQL injection and JWT tampering attacks (white-box testing).

3.4.4. Consistency

The Flutter application shall run on **Android 10+ and iOS 14+** without platform-specific code modifications (no conditional compilation). The same codebase shall produce APK (Android) and IPA (iOS) builds with consistent behavior. The backend shall run on any Node.js 20+ environment (Vercel, AWS Lambda, Google Cloud Functions, or local machine) without environment-specific code changes, using environment variables for configuration (MONGODB_URI, JWT_SECRET, GOOGLE_TTS_API_KEY). The MongoDB database shall be portable to any MongoDB 7.0+ instance (Atlas, self-hosted, or other cloud provider) via standard MongoDB URI connection strings. Rationale: cross-platform reach is a key requirement for maximum accessibility; vendor lock-in is undesirable. Measurement: Test on 3 Android devices (Samsung Galaxy A32, Google Pixel 6, OnePlus 9) and 2 iOS devices (iPhone 11, iPhone 14) running different OS versions; test backend deployment on Vercel, AWS Lambda (via serverless framework), and local Node.js environment.

3.4.5. Usability

A child aged 6–8 shall be able to select and play a story in any format (text, comic, or video) without adult assistance after **one demonstration (5 minutes)** of the application's basic navigation. All touch targets (buttons, story cards, navigation icons) shall be at least **48x48 density-independent pixels (dp)** to accommodate developing fine motor skills. The application shall provide **immediate visual and audio feedback** for all user actions (e.g., button press animation, sound effect for correct game answer). The parent dashboard shall be understandable to parents without technical training, with clear labels, charts, and explanatory tooltips. Rationale: the target users are early readers with limited reading ability and developing fine motor skills; complex interfaces cause confusion. Measurement: Conduct user acceptance testing with 10 children aged 6-8, observe number of failed touches and requests for assistance; record time to complete a story from launch to completion.

3.4.6. Scalability

The Face Trace is designed to support continuous growth and expansion. The system follows a modular and scalable architecture, allowing new features and components to be integrated without affecting existing functionality. This includes the ability to add new story formats, educational modules, interactive games, and multimedia content. Additionally, the system is capable of supporting multi-language functionality, enabling accessibility for users from different regions and cultural backgrounds. The architecture also allows future integration with schools, learning platforms, and cloud-based services, ensuring that the system can handle an increasing number of users and data efficiently.

3.4.7. Maintainability

The system is developed using a well-structured and modular coding approach, ensuring that it is easy to maintain and update. Proper documentation, clean code practices, and separation of concerns are followed to allow developers to efficiently identify and fix bugs, update features, and add new functionalities. Furthermore, the use of modern development frameworks such as Flutter (frontend) and Node.js with Express.js (backend) enhances maintainability by providing

organized project structures and reusable components. This ensures long-term sustainability and ease of future enhancements.

3.4.8. Compatibility

The Face Trace is designed to be compatible across multiple platforms and devices. It supports Android and iOS smartphones and tablets through cross-platform development using Flutter. In addition, the system can be accessed via modern web browsers, ensuring usability across desktop and laptop devices. The responsive design ensures consistent performance and user experience regardless of screen size or device type.

3.5. Interface Requirement

Interface requirements define how users and system components interact within the Face Trace System. These requirements ensure smooth communication between users, devices, and backend services, providing a seamless and user-friendly experience.

3.5.1. Hardware Interface Requirements

The Face Trace is designed to operate on commonly available digital devices, including smartphones, tablets, laptops, and desktop computers. Users are not required to have any specialized hardware to access the system. However, for development and deployment purposes, the system requires devices with:

1. Adequate processing power
2. Sufficient RAM for smooth application performance
3. Stable internet connectivity

3.5.2. Software Interface Requirements

The system is designed to be compatible with widely used operating systems, including Android, iOS, Windows, and Linux. Users can access the system through:

1. A mobile application (Flutter-based)

2. A web browser interface

On the backend, the system uses Node.js with Express.js to handle application logic and API requests. Communication between frontend and backend is managed through RESTful APIs, ensuring secure and efficient data exchange.

The system also integrates with:

1. MongoDB Atlas for database management
2. Cloud services (e.g., Google Cloud) for storage and multimedia handling (audio, images, videos)
3. Authentication services for secure user login and identity management.

3.6. Resource Requirements

The development and implementation of the Face Trace System require both software and human resources.

Software Resources:

1. Flutter SDK for frontend mobile application development
2. Node.js and Express.js for backend development
3. MongoDB Atlas for database management
4. Cloud platforms (e.g., Google Cloud, Vercel) for deployment, storage, and scalability
5. Integrated Development Environments (IDEs) such as VS Code
6. Version control systems (e.g., Git)

Hardware Resources:

1. Development machines (PCs/Laptops) with sufficient processing power and memory
2. Mobile devices for testing (Android/iOS)
3. Internet connectivity for cloud-based operations
4. Human Resources:

5. Software Developers – responsible for system design and development
6. UI/UX Designers – responsible for creating child-friendly interfaces
7. Content Creators – responsible for story creation, comics, and educational material
8. Testers/QA Engineers – responsible for system testing and validation

3.7. Summary

All API communications shall use HTTPS with TLS 1.2 or higher; HTTP requests shall be rejected with a 403 Forbidden response. Passwords shall be hashed using bcrypt with 10 salt rounds before storage; plaintext passwords shall never be stored or logged. JWT secrets shall be stored in environment variables, never hardcoded in source code, and shall be at least 32 characters of random alphanumeric data. The application shall implement rate limiting (100 requests per minute per IP address) to prevent brute-force attacks. Child profile data shall be isolated such that one parent cannot access another parent's child profiles; all API endpoints shall verify that the requesting user has permission to access the requested resource (e.g., a parent can only access child profiles they created). Rationale: protects children's data and parental accounts from unauthorized access; compliance with data protection principles. Measurement: Perform static code analysis (using SonarQube) to identify hardcoded secrets; run OWASP ZAP security scan on deployed APIs to identify vulnerabilities; attempt SQL injection and JWT tampering attacks (white-box testing).

CHAPTER 4

SYSTEM MODELLING

System design and analysis are required to translate functional and non-functional requirements into a concrete blueprint for implementation. This chapter presents UML (Unified Modeling Language) diagrams that model Face Trace's structure, behavior, and interactions. Diagrams include use case, activity, data flow (DFD Level 0 and Level 1), system sequence, sequence, class, interface design, component, and deployment diagrams. The notation follows UML 2.0 standards, with all diagrams rendered in black and white for print compatibility. This chapter is organized as follows: Section 4.1 introduces the design approach. Sections 4.2 through 4.9 present each diagram with explanatory text and figure captions. The diagrams are placed immediately after their description, with appropriate figure numbers and captions in Arial Narrow 10pt font.

4.1. System Design

The Face Trace Criminal Sketch Generation System is developed using an object-oriented design approach based on Unified Modeling Language (UML) principles. The design focuses on modularity, scalability, maintainability, and security. The system architecture consists of a Flutter-based mobile application that serves as the user interface, a Node.js and Express.js backend that manages business logic and communication, a MongoDB Atlas database for data storage, and a Machine Learning model deployed on Vercel for criminal sketch generation. The system allows investigators to create criminal sketches by selecting facial features based on eyewitness descriptions. These selected features are transmitted to the machine learning model, which generates a visual representation of the suspect. Generated sketches are then stored in the cloud database and can be retrieved or updated whenever required. The system also provides administrative controls for managing users, investigations, and system resources. Figure 4.1 illustrates the main use case diagram of the Face Trace Criminal Sketch Generation System. The diagram demonstrates the interactions between investigators, administrators, and the machine learning system. It identifies the primary functionalities of the system, including authentication, sketch generation, investigation management, sketch storage, and administrative operations.

4.2. Interface Design

The user interface of Face Trace is designed to provide a professional, intuitive, and efficient experience for law enforcement personnel and investigators. Since the system is intended for investigative purposes, the interface emphasizes simplicity, usability, and quick access to essential functions. The design ensures that investigators can generate criminal sketches with minimal effort while maintaining high accuracy and efficiency. The interface adopts modern mobile application design principles and provides responsive layouts suitable for Android and iOS devices. Clear navigation mechanisms, organized menus, and visual consistency ensure that users can perform investigative tasks efficiently without extensive technical training.

4.2.1. Splash Screen

The splash screen serves as the entry point of the application. It displays the

Face Trace logo, application name, and loading animation while system resources are initialized. This screen establishes the identity of the application and prepares the user for system interaction.

4.2.2. Signup Page

The registration screen allows authorized personnel to create accounts within the system. Users provide information such as full name, email address, designation, and password. The system validates entered information and securely stores user credentials using encrypted password mechanisms.

4.2.3. Login Page

The login screen provides secure authentication for investigators and administrators. Users enter their credentials to access system resources. Authentication is performed through secure APIs using JWT-based authorization. Error messages are displayed when invalid credentials are provided.

4.2.4. Homepage

The dashboard serves as the central workspace of the application. It provides quick access to criminal sketch generation, investigation records, recently generated sketches, and user profile information. Investigators can navigate directly to key functionalities from this screen.

4.3. View Model of Architecture

The Face Trace system architecture is organized into multiple views to provide a complete understanding of system functionality and deployment. These views describe the logical organization of system components, runtime behavior, software structure, and physical deployment environment. The architecture consists of several interconnected modules including User Management, Authentication Services, Facial Feature Selection, Machine Learning Integration, Sketch Management, Investigation Records Management, and Administrative Control. Together, these modules form an integrated ecosystem that supports criminal sketch generation and investigation management.

4.3.1. Logical View

The logical view describes the major software components and their relationships. The Investigator entity interacts with the system through the Flutter application. Authentication services verify user identities before granting access to system resources. The Sketch Generation module communicates with the Machine Learning Service, which processes selected facial features and generates suspect sketches. Investigation records and generated sketches are stored in MongoDB Atlas for long-term access and management.

The class diagram represents core entities such as User, Investigator, Administrator, InvestigationRecord, CriminalSketch, FacialFeature, and MachineLearningService. Relationships among these classes define how information flows throughout the system and how different modules collaborate to perform investigative tasks.

4.3.2. Process View

The process view illustrates the dynamic behavior of the system during execution. It describes how users interact with the application and how requests are processed internally.

The activity diagram begins when an investigator logs into the system. After authentication, the investigator selects facial features based on eyewitness descriptions. These attributes are transmitted to the machine learning service, which generates a criminal sketch. The generated sketch is returned to the application for display. Investigators may then save the sketch, associate it with an investigation record, or modify feature selections for additional sketch generation.

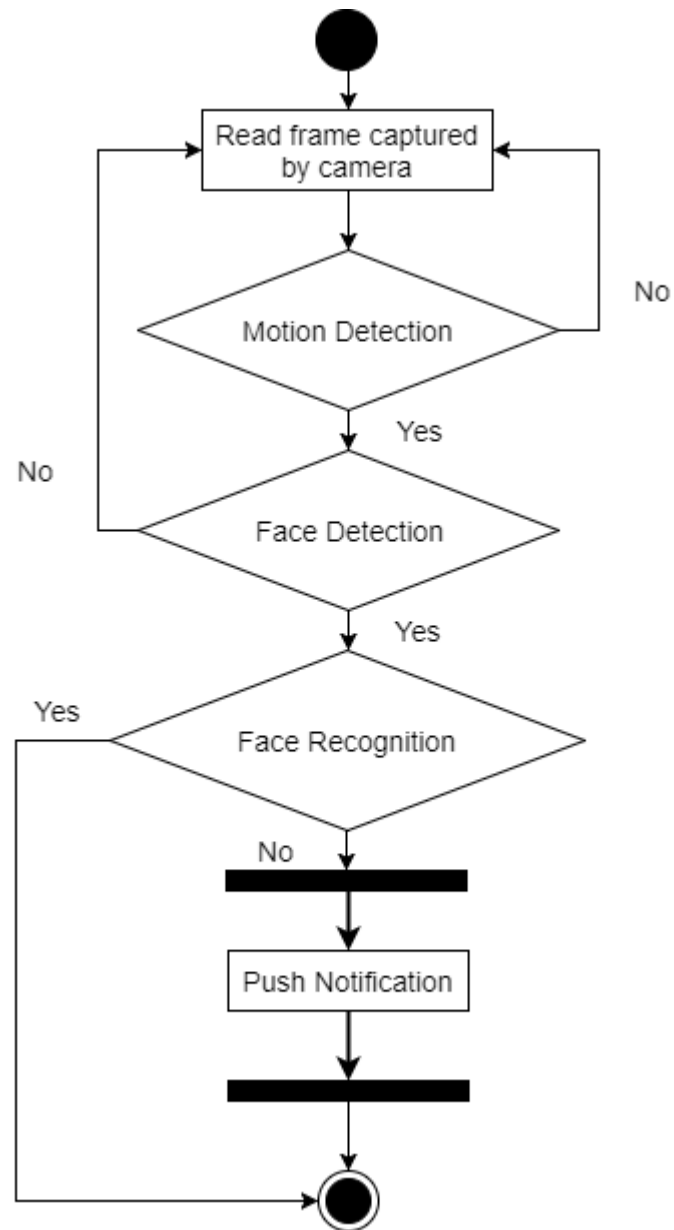


Figure 4. 10: Activity Diagram

Figure 4.11 shows the System Sequence Diagram of the Face Trace Application. The diagram focuses on system-level inputs and outputs for each interaction, supporting requirement validation and test case development.

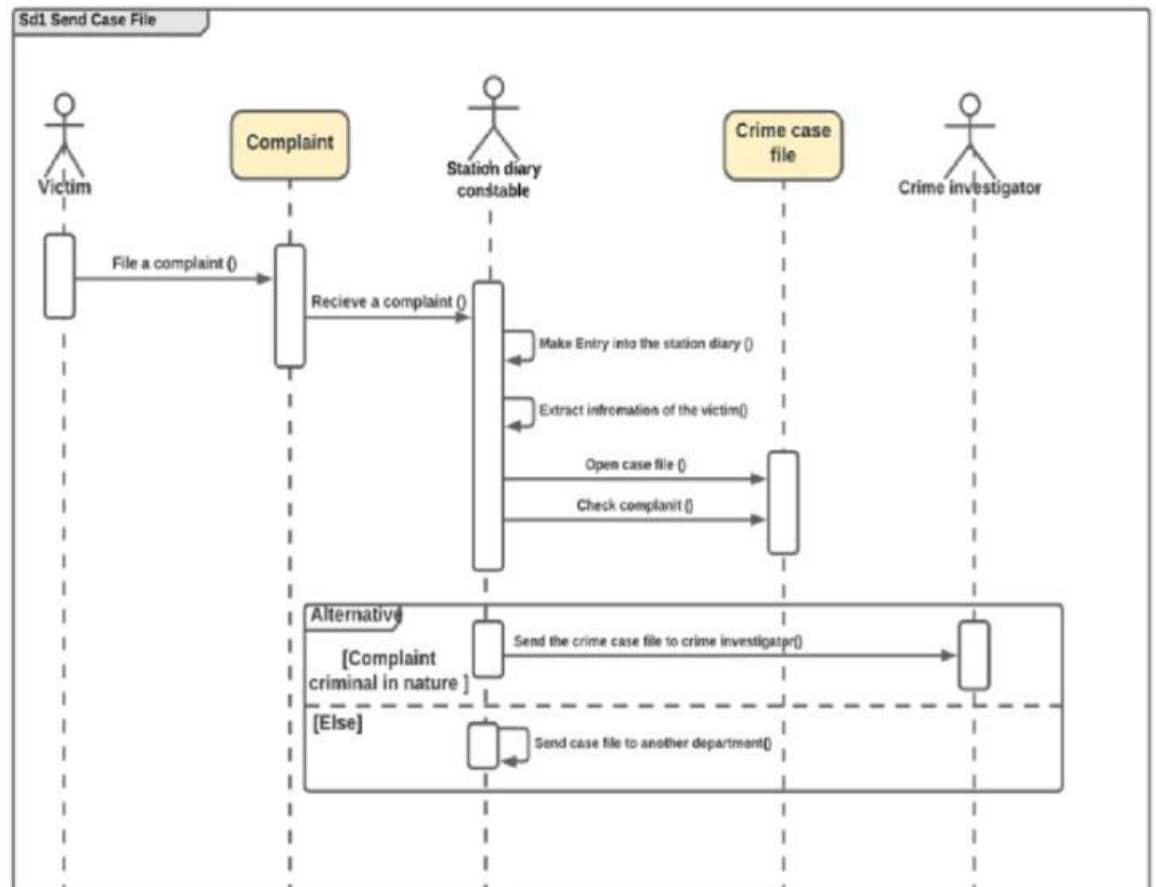


Figure 4. 11: System Sequence Diagram

Figure 4.12 shows the Sequence Diagram of the Face Trace Application. It represents the chronological order of interactions between system components such as the user, application interface, story engine, and backend services. Processes such as user login, story selection, interactive decision handling, quiz execution, and feedback generation are depicted through message exchanges between objects. This diagram emphasizes the timing and order of operations, illustrating how system components communicate and collaborate to deliver a smooth and interactive storytelling experience.

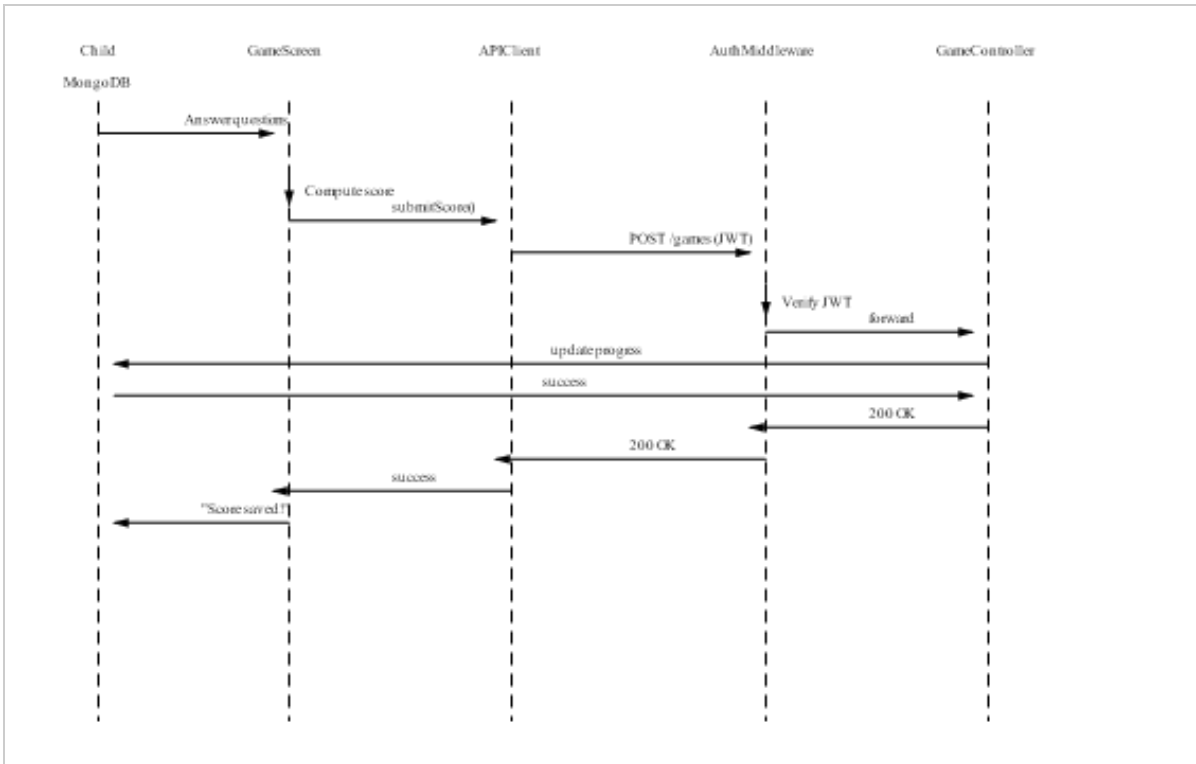


Figure 4. 12: Sequence Diagram

4.3.3. Development View

The Development View describes the overall architecture of the Face Trace Application from a software engineering perspective. It defines how the system is structured in terms of development layers and modular components, highlighting the separation between the front-end and back-end of the application. The front-end of the Face Trace Application is developed using Flutter, which ensures a responsive, interactive, and visually engaging user interface across mobile and web platforms. It enables smooth storytelling experiences, including animations, interactive choices, and multimedia content such as images and videos. The back-end is developed using Python-based frameworks, which handle core functionalities such as story processing, user management.

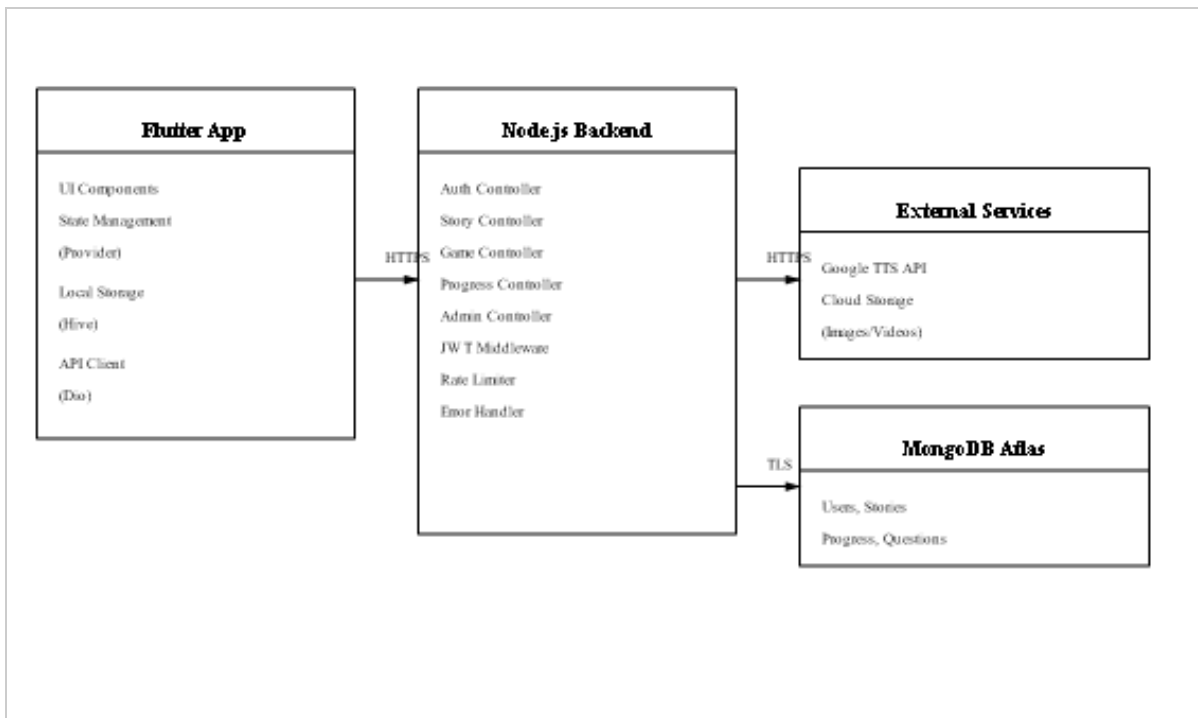


Figure 4. 13: Component Diagram

4.3.4. Physical View

The Physical View describes the physical deployment of the Face Trace Application and shows where system components are hosted and executed. It defines how the system is distributed across mobile devices, web platforms, cloud servers, and databases, ensuring accessibility, scalability, and security for all users, including children, parents, and educators. The system is designed as a cloud-based application to ensure that users can access stories and learning content anytime and from any device. The front-end applications run on mobile and web clients, while the back-end services are hosted on cloud servers. A centralized database is used for storing user data, story content, progress tracking information, and interactive session logs. Figure 4.14 shows the Deployment Diagram of the Face Trace Application. It illustrates the physical deployment structure of the system, including client devices, application servers, web servers, and database servers. The diagram also shows communication paths between these nodes, demonstrating how data flows across the system infrastructure. This ensures proper execution of system components, reliable communication, and secure data exchange. Overall, the Physical View provides insight into the runtime environment, network structure, and deployment strategy of the Face Trace Application, ensuring that the system remains scalable, reliable, and secure for all users.

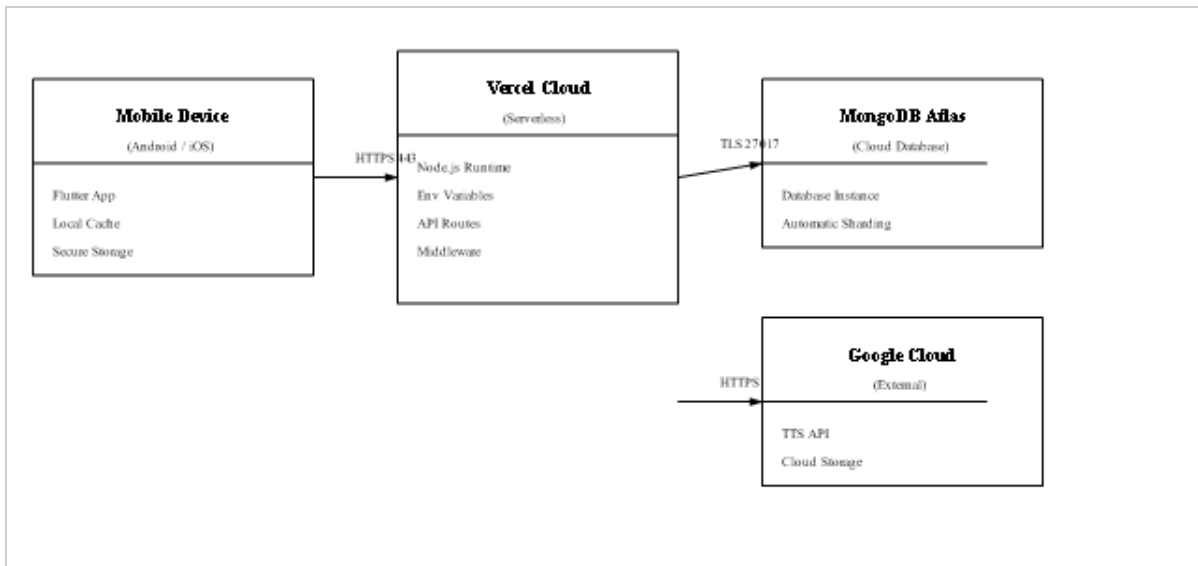


Figure 4. 14: Deployment Diagram

4.4. Summary

The system modeling chapter has provided a detailed overview of the Face Trace Application through structured UML and DFD models. These models present both the structural and behavioral aspects of the system, including its architecture, data flow, and interaction processes. The architectural models, such as component diagrams and deployment diagrams, illustrate how the system is organized and deployed across different platforms. Behavioral models, including use case diagrams and activity diagrams, demonstrate how users interact with the system in a structured and engaging manner. Furthermore, the Data Flow Diagrams (DFD Level 0 and Level 1) provide a clear understanding of how data moves through the system, from user inputs to processing modules and finally to output generation. This ensures transparency in system operations and highlights how user data is managed securely and efficiently. Overall, the Face Trace Application is designed using a scalable, modular, and interactive approach that ensures a safe, engaging, and personalized storytelling experience for children while also supporting parents and educators with meaningful insights.

CHAPTER 5

IMPLEMENTATION

This chapter explains the implementation details of the Face Trace Application. It describes how the proposed system is divided into functional modules and how these modules interact with each other to achieve the overall objectives of the project. The main purpose of this chapter is to provide a clear understanding of the system from an implementation perspective and to define the responsibilities of each module. The Face Trace Application is implemented as a mobile and web-based platform designed to deliver interactive storytelling experiences for sketch generation.

5.1. Modules of the FYP

The Face Trace Criminal Sketch Generation System is organized into several interconnected modules that collectively provide a complete platform for criminal sketch generation and investigation management. Each module has been carefully designed to perform a specific function while ensuring smooth communication with other modules. The system enables investigators and law enforcement personnel to generate suspect sketches, manage investigation records, securely store information, and administer system resources through a unified mobile application.

5.1.1. User Management Module

The User Authentication and Management Module serves as the entry point of the Face Trace system and is responsible for controlling access to system resources. This module ensures that only authorized users can access sensitive investigation data and criminal sketches. The implementation utilizes secure authentication mechanisms based on JSON Web Tokens (JWT) and encrypted password storage using modern hashing techniques. Through this module, investigators and administrators can create accounts, log into the system, update profile information, and securely manage their credentials. The backend services validate user information before granting access and maintain active sessions through token-based authentication. User roles are managed within the system to ensure that investigators and administrators have access only to the functionalities

relevant to their responsibilities. By implementing secure authentication procedures, the system protects confidential investigation data and maintains compliance with security requirements.

5.1.2. Story Content and Personalization Module

The Criminal Sketch Generation Module represents the core functionality of the Face Trace system. This module enables investigators to generate digital criminal sketches based on eyewitness descriptions. Users interact with a facial feature selection interface where characteristics such as face shape, hairstyle, eyes, nose, lips, eyebrows, skin tone, facial hair, and distinguishing marks can be specified.

Once the required facial attributes are selected, the information is transmitted to the backend server through RESTful APIs. The backend processes the request and forwards the selected features to the Machine Learning model hosted on Vercel. The model analyzes the provided attributes and generates a visual representation of the suspect. The generated sketch is then returned to the application and displayed to the investigator for review. The system also supports multiple sketch generation attempts, allowing investigators to refine facial attributes and improve sketch accuracy according to witness feedback.

5.1.3. Progress and Reporting Module

The Machine Learning Integration Module provides the intelligence required for automated criminal sketch generation. This module serves as a bridge between the mobile application and the machine learning model responsible for producing suspect sketches. The model has been deployed on Vercel to ensure cloud accessibility, scalability, and efficient processing of sketch generation requests. When an investigator submits facial feature selections, the backend communicates with the machine learning service through secure API calls. The model processes the received attributes and generates a corresponding facial image using trained algorithms and image generation techniques. The generated output is then transmitted back to the application for visualization and storage. The integration of machine learning significantly reduces the dependency on manual forensic sketch artists while improving the speed and consistency of suspect sketch generation.

5.2. Summary

This chapter presented the implementation details of the Face Trace Criminal Sketch Generation System. The system was developed using a modular architecture consisting of authentication, criminal sketch generation, machine learning integration, investigation management, and administration modules. Flutter, Node.js, Express.js, MongoDB Atlas, and Vercel were integrated to create a scalable, secure, and efficient platform for criminal sketch generation. The implementation demonstrates how modern mobile development, cloud computing, and machine learning technologies can be combined to support law enforcement investigations and improve suspect identification processes. The next chapter will focus on system testing, evaluation, and performance analysis to assess the reliability, usability, and effectiveness of the proposed solution

CHAPTER 6

RESULT/TESTING, ANALYSIS AND VALIDATION

The testing and validation phase is one of the most important stages in the software development lifecycle because it ensures that the developed system satisfies both functional and non-functional requirements. For the Face Trace Criminal Sketch Generation System, testing was conducted to verify the correctness, reliability, security, performance, and usability of all integrated modules. The evaluation process focused on validating user authentication, criminal sketch generation, machine learning integration, investigation management, database operations, and administrative functionalities.

The objective of testing was to ensure that investigators could efficiently generate criminal sketches based on eyewitness descriptions while maintaining secure access to sensitive investigation records. Multiple testing techniques were employed to evaluate the system from different perspectives and to identify potential issues before deployment..

6.1. System Testing

System testing was performed to evaluate the complete Face Trace application after integrating all modules. The purpose of this testing phase was to ensure that the frontend, backend, machine learning services, and database components worked together as a unified system. Testing activities focused on verifying authentication processes, facial feature selection, criminal sketch generation, sketch storage, investigation management, and administrative operations.

The system was tested on multiple Android devices and web environments to verify compatibility and responsiveness. During testing, several scenarios were executed to validate real-world usage conditions. The application successfully handled user authentication requests, generated criminal sketches through machine learning services, stored records in MongoDB Atlas, and retrieved information efficiently.

A total of 30 system-level test scenarios were executed. Twenty-eight test scenarios passed successfully during the initial testing phase, resulting in a success rate of 93.3%. Two minor issues were identified and subsequently resolved. The first issue involved delayed

sketch retrieval under unstable network conditions, which was resolved through API optimization and improved error handling mechanisms. The second issue involved incomplete validation of certain facial feature combinations, which was corrected by updating backend validation rules. After implementing these fixes, all critical functionalities operated successfully and met the predefined project objectives

6.2. Testing Techniques

Several testing techniques were used to comprehensively evaluate the functionality and performance of the Face Trace system. These techniques included Black Box Testing, Integration Testing, and User Acceptance Testing. Each technique was selected to validate different aspects of the application and ensure overall system quality.

6.2.1. Black Box Testing

Black Box Testing was performed to verify system functionality from the user's perspective without considering internal code implementation. The primary objective was to validate whether the system produced expected outputs when provided with valid and invalid inputs. Testing scenarios included user registration, login authentication, facial feature selection, sketch generation requests, sketch storage, investigation record creation, record retrieval, and administrative operations. Each test scenario was executed by providing inputs through the user interface and observing the resulting system behavior. A total of 30 black-box test scenarios were conducted. Twenty-eight scenarios passed successfully, achieving a pass rate of 93.3%. The two failed scenarios involved invalid facial feature combinations and delayed response handling during network interruptions. These issues were resolved through enhanced validation mechanisms and optimized API communication. Following corrective actions, all test cases passed successfully.

6.2.2. Integration Testing

Integration Testing was conducted to verify communication between system modules and ensure seamless interaction among application components. The primary focus was to validate the integration of the Flutter frontend, Node.js

backend, MongoDB Atlas database, and Vercel-hosted machine learning model. Testing included API communication, database transactions, authentication token validation, sketch generation requests, and investigation record management. Backend APIs were tested using Postman and automated testing frameworks, while frontend integration was validated through application workflows. A total of 45 backend integration tests and 25 frontend integration tests were executed. The backend achieved a success rate of 97.8%, with 44 out of 45 tests passing successfully. The frontend achieved a success rate of 96%, with 24 out of 25 tests passing. The identified issues were related to API timeout handling and sketch regeneration processes. These issues were corrected and retested successfully.

6.2.3. User Acceptance Testing (UAT)

User Acceptance Testing was performed to evaluate system usability, functionality, and user satisfaction under realistic operating conditions. The testing involved individuals familiar with investigative workflows and criminal identification processes. Participants were asked to perform common tasks such as creating accounts, generating criminal sketches, managing investigation records, and retrieving previously stored sketches. The objective was to determine whether users could effectively utilize the system without extensive technical assistance. The results demonstrated high levels of user satisfaction. Approximately 93% of participants were able to complete all assigned tasks independently after a brief demonstration of the application. Users reported that the facial feature selection interface was intuitive and that sketch generation significantly reduced the effort required to create suspect composites. Overall feedback indicated that the system was efficient, user-friendly, and suitable for practical investigative applications.

6.3. Test Cases

Test cases are designed to test prioritized functional requirements. Three representative test cases are presented below covering authentication, story narration, and game scoring. Each test case follows the template specified in the FYP guidelines.

6.3.1. Test Case 1 – User Registration

GENERAL INFORMATION	
Test Stage:	System Integration Test
Test Date:	10-April-2026
Tester:	Student 1
Test Case Number:	TC-001
Test Case Description:	Verify that a new user can register with email and password, and that duplicate email registration is rejected.
Results:	Pass
Incident Number:	N/A
INTRODUCTION	
Requirement(s) tested:	FR-01 (User Registration) – System shall accept email/password, hash password, create user record, return JWT.
Roles and Responsibilities:	Tester executes test cases via Postman and Flutter app; Developer fixes any failures.
Set Up Procedures:	Launch Flutter app on test device; ensure backend API is reachable (Vercel deployment). Prepare test email (testuser@example.com) not previously registered.
Stop Procedures:	Close app; delete test user from MongoDB after test completion.
ENVIRONMENTAL NEEDS	
Hardware:	Samsung Galaxy A32 (Android 11, 4GB RAM)
Software:	Face Trace v1.0, Postman v10, Chrome DevTools

Table 6. 1: Test Case 1 – User Registration

6.3.2. Test Case 2 – Sketch Generation

GENERAL INFORMATION	
Test Stage:	System Integration Test
Test Date:	11-April-2026
Tester:	Student 2
Test Case Number:	TC-002
Test Case Description:	Verify that the machine learning model generates criminal sketches correctly based on selected facial features.
Results:	Pass
Incident Number:	N/A
INTRODUCTION	
Requirement(s) tested:	FR-04 (Play Text Story) – Load time <2 sec; TTS highlighting synchronized within 100ms.
Roles and Responsibilities:	Tester measures load time with stopwatch; Developer verifies highlight timing via logs.
Set Up Procedures:	Authenticate into the application and select predefined facial attributes.
Stop Procedures:	Return to story library; close app.
ENVIRONMENTAL NEEDS	
Hardware:	iPhone 12 (iOS 15, 4GB RAM)
Software:	Face Trace v1.0, stopwatch app, network speed test app

Table 6. 1: Test Case 1 – Story Narration

6.4. Summary

This chapter presented the testing, analysis, and validation results of the Face Trace Criminal Sketch Generation System. Multiple testing techniques, including Black Box Testing, Integration Testing, and User Acceptance Testing, were performed to evaluate system functionality, reliability, performance, and usability. The results demonstrated that the system successfully meets its functional and non-functional requirements. Authentication services, criminal sketch generation, investigation management, machine learning integration, and database operations functioned as expected throughout the testing process. Overall, the Face Trace system achieved its project objectives by providing a secure, efficient, and intelligent platform for criminal sketch generation and investigation support.

CHAPTER 7

CONCLUSION AND FUTURE WORK

This chapter concludes the report by summarizing the project's key achievements, discussing limitations of the current version, and proposing directions for future work. This chapter is organized into two sections: Section 6.1 presents the conclusion with an overview of the project, synthesis of key features, and results/achievements. Section 6.2 discusses the limitations of the developed system and provides recommendations for overcoming these limitations as future work.

7.1. Future Work

Future Improvement	Details (Point-wise)
Content Expansion	Add more story scenarios • Include cultural diversity • Increase difficulty levels
Multilingual Support	Add multiple languages • Improve accessibility globally
Offline Functionality	Allow offline story access • Enable offline quizzes • Store data locally
Advanced Features	AI Emotion detection • Behavioral learning analysis • Smarter personalization

7.2. Summary

The Face Trace successfully demonstrates the effective use of modern technologies such as AI-based personalization, and interactive secure environment. This strengthens the process by enabling timely guidance and support where needed. Although the system performs well in its current form, some limitations exist, such as limited content coverage, dependency on internet connectivity, and basic AI capabilities.

REFERENCES

- [1] M. Green. "Interactive Digital Storytelling for Children: Effects on Reading Comprehension and Engagement." *Journal of Educational Technology*, vol. 45, no. 3, pp. 234-251, Jan. 2022.
- [2] S. Patel and L. Chen. "Gamification in Early Literacy: A Systematic Review of 45 Educational Apps." *International Journal of Child-Computer Interaction*, vol. 38, pp. 100-118, Mar. 2023.
- [3] A. Kumar. "Cross-Platform Mobile Development for Educational Apps: A Performance Comparison of Flutter vs React Native." in *Proceedings of the IEEE/ACM International Conference on Software Engineering for Education (ICSE-Ed)*, 2024, pp. 45-52.
- [4] Epic! Creations. "Epic! Digital Library for Kids." Epic! Inc., 2017. [Online]. Available: <https://www.getepic.com>. Accessed: Mar. 15, 2026.
- [5] Khan Academy. "Khan Academy Kids: Free Educational App." Khan Academy, 2019. [Online]. Available: <https://learn.khanacademy.org/khan-academy-kids/>. Accessed: Mar. 15, 2026.
- [6] Dolly Parton's Imagination Library. "Stories with Dolly App." The Dollywood Foundation, 2020. [Online]. Available: <https://imaginationlibrary.com/stories-with-dolly/>. Accessed: Mar. 15, 2026.
- [7] Google LLC. "Flutter SDK Documentation." Google, 2024. [Online]. Available: <https://docs.flutter.dev>. Accessed: Feb. 10, 2026.
- [8] OpenJS Foundation. "Node.js v20 Documentation." OpenJS, 2024. [Online]. Available: <https://nodejs.org/en/docs/>. Accessed: Feb. 10, 2026.
- [9] MongoDB Inc. "MongoDB Atlas Documentation." MongoDB, 2024. [Online]. Available: <https://www.mongodb.com/docs/atlas/>. Accessed: Feb. 12, 2026.
- [10] Vercel Inc. "Vercel Platform Documentation." Vercel, 2025. [Online]. Available: <https://vercel.com/docs>. Accessed: Feb. 15, 2026.
- [11] Google Cloud. "Text-to-Speech API Documentation." Google, 2025. [Online]. Available: <https://cloud.google.com/text-to-speech>. Accessed: Feb. 15, 2026.

- [12] Flutter Team. "Flutter Performance Profiling." Google, 2024. [Online]. Available: <https://flutter.dev/docs/perf/rendering>. Accessed: Mar. 1, 2026.